Mantık Devreleri - Ödev 1

Deney yaprakları 1-7

Efe Özan - 170425062

12.10.2026

Deneylerin hesapları burada. İlk deneyin programı: `sayi_donusturucu.py`.

İlk deneyin kontrolü: `dogrulama_1.py`. Diğer deneylerin kontrol kodları da 2'den 7'ye kadar ayrı Python dosyalarında.

Deney 1 - Sayı sistemi dönüştürücüsü

1. Tam kısmı hedef tabana bölerek çevirdim. İlk çıkan kalan en sağdaki rakam olduğu için kalanları ters yazmak gerekiyor. Kesirde ise çarpma yapılıyor ve çıkan rakamlar sırayla yazılıyor. Mesela 45'i 16'ya bölünce önce D sonra 2 kalıyor, yani `2D`. Kesir olan 0.40625'ten de sırayla 6 ve 8 geliyor. Sonuç `2D.68`.

2. Programda 2, 8, 10 ve 16 tabanları var. Boş bir şey yazılırsa, sayı negatifse, iki nokta varsa veya rakam tabana uymuyorsa hata veriyor. Örneğin ikili tabanda `102`, sekizli tabanda `128` yazılamaz.

3. Kesirli kısımda en fazla 8 rakam yazdırdım. Kalan 0 olursa = koyuyorum. Sekiz rakamdan sonra hâlâ kalan varsa ≈ koyuyorum, çünkü sayı tam bitmemiş oluyor. Örneğin 0.1 sayısında durum bu.

4. Kod `sayi_donusturucu.py` dosyasında. Taban değiştirme işinde hazır `bin`, `oct`, `hex` fonksiyonlarını kullanmadım.

5. Şu girişleri denedim. Hatalı olanlar için sayı yerine hata mesajı çıkıyor.

Girdi	Taban	Beklediğim	Programın sonucu
`101101.01101`	2 → 16	`2D.68`	`2D.68`
`0`	10 → 2	`0`	`0`
`.101`	2 → 10	`0.625`	`0.625`
`001011.10`	2 → 10	`11.5`	`11.5`
`0.1`	10 → 2	`0.00011001` (yaklaşık)	`0.00011001` (yaklaşık)
`2D.68`	16 → 8	`55.32`	`55.32`
`102`	2 → 10	Hata	Hata
`128`	8 → 2	Hata	Hata
Boş girdi	10 → 2	Hata	Hata
`10.11.01`	2 → 16	Hata	Hata

Örnek sayıyı programdan ayrı olarak da hesapladım:

`101101.01101` (2) = 32 + 8 + 4 + 1 + 1/4 + 1/8 + 1/32 = **45.40625**.

`2D.68` (16) = 2 × 16 + 13 + 6/16 + 8/256 = **45.40625**.

Düşünme sorusu: Hayır, geri dönünce aynı sayıyı bulmak yetmez. Program C yerine D yazıyorsa ve geri okurken de aynı şekilde karıştırıyorsa yanlış bir sonuçtan yine eski sayıya dönebilir. Mesela `2D.68` yerine `2C.68` yazıp sonra onu hatalı okuyabilir. Bu yüzden aradaki sayıya da bakmak gerekiyor.

Deney 2 - Sınırlı bit sayısıyla gösterim

1-2. 0.125 = 1/8, yani kesir tarafında 3 bit lazım. Tam tarafında 1000'i yazmak için 10 bit lazım. Çünkü 9 bit en fazla 511 gösteriyor. Böylece toplam **13 bit** çıkıyor.

3. 12 bit yetmiyor. Kesir için 3 bit ayırınca tam tarafa 9 bit kalıyor ve en fazla 511.875 yazılabiliyor. Bir de sayı adedini saydım: 1000 / 0.125 + 1 = **8001**. Ama 12 bit ile yalnızca 4096 farklı değer var.

4-5. Noktanın solunu 10, sağını 3 bit aldım. Aynı uzunlukta yazınca sayılar şöyle oldu:

- 0 = `0000000000.000`
- 0.125 = `0000000000.001`
- 173.625 = `0010101101.101`
- 1000 = `1111101000.000`

6. 173.625 sayısı sekizlide `255.5`, onaltılıda `AD.A`. Kontrol etmek için onluğa geri çevirdim: 2 × 64 + 5 × 8 + 5 + 5/8 = **173.625**, ayrıca 10 × 16 + 13 + 10/16 = **173.625**.

Düşünme sorusu: Hayır. Çünkü 0.1 = 1/10 ve 10'un içinde 5 çarpanı var. İkili kesirlerin paydaları 2'nin kuvvetlerinden geliyor. O yüzden 0.1'i biten bir ikili kesir olarak yazamıyoruz.

Deney 3 - Sıfır ekleme ve gruplandırma

1. `101101.01101` sayısında noktadan başlayıp sola ve sağa doğru grupladım. Köşeli parantezdeki sıfırlar sonradan eklediklerim.

- Sekizli: `101 101 . 011 01[0]` → `55.32`
- Onaltılı: `[00]10 1101 . 0110 1[000]` → `2D.68`

2. Tam kısmın başındaki sıfırlar ve kesrin sonundaki sıfırlar toplama 0 ekliyor. Var olan rakamların basamakları değişmediği için sayının değeri de değişmiyor.

3. Kesri `01101` yerine `00001101` yazınca sonuç `2D.0D` oluyor. Onluk değeri 45 + 13/256 = **45.05078125**. Doğru sayı 45.40625 olduğu için aradaki fark **0.35546875**.

4. Her zaman sola sıfır eklenmez. Mesela `0.1` ikilide 0.5 eder. `0.0001` yazarsak 0.0625 olur. Değişmemesi için `0.1000` yazmamız gerekir.

5. `101101.01101` (2), `55.32` (8) ve `2D.68` (16) sayılarını ayrı ayrı açınca hepsinin değeri **45.40625** çıktı.

Düşünme sorusu: Evet, ikisi de 5.5. Ama `00101.100` için 8 bit, `101.1` için 4 bit yazılmış. Yani aynı sayının farklı uzunlukta yazılması mümkün.

Deney 4 - Kesme ve yuvarlama

1-3. Kesirleri ikiliye çevirdim. 8 basamakta bitmeyenleri orada kestim. Sonra bir de en yakın 8 basamaklı sayıya yuvarladım. Hataları da çıkan sayıları onluğa geri çevirerek buldum.

Sayı	8 bitte kesme	Kesme hatası	Yuvarlama	Yuvarlama hatası
0.625	`0.10100000`	0	`0.10100000`	0
0.1	`0.00011001`	0.00234375	`0.00011010`	0.0015625
0.65	`0.10100110`	0.0015625	`0.10100110`	0.0015625
0.975	`0.11111001`	0.00234375	`0.11111010`	0.0015625

4. 0.625 tam çıkıyor. Diğer üç sayıda kesir devam ediyor. O yüzden onlar için = yerine ≈ yazılmalı.

5. Biten bir ikili kesrin paydası 2'nin kuvveti oluyor. Mesela `0.101` (2) = 5/8. Paydada 5 gibi başka bir çarpan kalırsa bitmiyor.

Düşünme sorusu: Aynı değiller. İkili sistemde 8. basamak 1/256 değerinde. Onaltılıda ise 8. basamak 1/4294967296. Yani onaltılıda daha küçük farkları yazabiliriz.

Deney 5 - Yanlış ifadeleri kontrol etme

A: `0.101` (2) = 1/2 + 1/8 = **0.625**. 0.65 değil, 0.025 eksik. = yanlış.

B: `0.F99` (16) = 15/16 + 9/256 + 9/4096 = **0.974853515625**. Burada da 0.975 ile küçük bir fark var: **0.000146484375**.

C: Kesrin yanlış tarafına sıfır konduğu için `2D.0D` bulunmuş. Ama bu 45.05078125 ediyor. Doğrusu **`2D.68`**, yani 45.40625.

D: Tam sayının sağına sıfır eklemek sayıyı ikiye katlıyor. `101` ikilide 5, `1010` ise 10. O yüzden söylenen kural yanlış.

Düzeltilmiş hâlleri: A ve B için ≈ kullanılmalı. C'de `2D.68` yazılmalı. D'de ise "sağa sıfır eklemek tam sayıyı 2 ile çarpar" denmeli.

Düşünme sorusu: Olabilir. A'da 0.101 rakamları doğru bulunmuş ama sonrasında kesir devam ediyor. İşlemi erken bırakıp = yazınca sonuç yanlış oluyor.

Deney 6 - Kaybolan tabanları bulma

1-2. Her kaydı 2, 8, 10 ve 16 tabanında okudum. Yazılamayan yerlere çizgi koydum.

Kayıt	2 tabanı	8 tabanı	10 tabanı	16 tabanı
A: `17`	-	15	17	23
B: `21`	-	17	21	33
C: `10001`	17	4097	10001	65537

3-4. Üçünde de görünen tek sayı **17**. O yüzden A'nın tabanı 10, B'nin 8, C'nin 2. Diğer tabanları eşleştirince ortak başka sayı çıkmıyor.

5. Üç cihazda da `10` yazsaydı hangi tabanda olduklarını bulamazdık. Çünkü hepsi taban 2'de de, 8'de de, 10'da da, 16'da da aynı sayıyı gösterebilirdi. **4 ihtimal** kalırdı.

6. Sayının başına tabanını yazardım. Mesela `10#17`, `8#21`, `2#10001`. Böylece sonradan sayı okunurken tabanı karışmaz.

Düşünme sorusu: Hayır. Mesela `10001` dört tabanda da geçerli bir yazım. Hangi tabanda kaydedildiği sadece rakamlara bakarak anlaşılamaz.

Deney 7 - Tek bit hatası

1. 173'ün ikili yazılışı `10101101`. Soldan ilk bit 7 (MSB), en sağdaki 0 (LSB). Onaltılı karşılığı `AD`.

2-3. Her seferinde başlangıçtaki sayıyı alıp sadece bir biti değiştirdim:

Bit	Yeni ikili sayı	Onluk	Onaltılı	Değişim	Hata
7	`00101101`	45	2D	Azaldı	128
6	`11101101`	237	ED	Arttı	64
5	`10001101`	141	8D	Azaldı	32
4	`10111101`	189	BD	Arttı	16
3	`10100101`	165	A5	Azaldı	8
2	`10101001`	169	A9	Azaldı	4
1	`10101111`	175	AF	Arttı	2
0	`10101100`	172	AC	Azaldı	1

4. MSB değişince sayı her zaman artmıyor. Burada o bit 1'di. Sıfıra çevirince 128 azaldı ve **45** oldu.

5. 0-200 aralığının dışına sadece **237** çıktı. Diğer 7 sayı yanlış olsa bile aralıkta kaldığı için bu kontrol onları yakalayamaz. Mesela 45 de bunlardan biri.

6. `AD` sayısının birer bitini değiştirince sırasıyla `2D`, `ED`, `8D`, `BD`, `A5`, `A9`, `AF`, `AC` çıkıyor.

Düşünme sorusu: Tek bir bit değişse bile fark büyük olabilir. Sağdaki bit 1, soldaki bit 128 değerinde. Yani hangi bitin değiştiği önemli. k numaralı bit değişirse fark 2'nin k. kuvveti kadar olur.

YZ kullanımı: Çözümleri yazarken YZ'den yardım aldım. Sayıları kontrol etmek için de Python kullandım.